

Что такое изменение?

Это контролируемое и задокументированное обновление системы. Каждое изменение должно иметь:

- **Идентификатор** (уникальный номер)
- **Дату** внесения
- **Ответственного** человека
- **Описание** того, что меняется
- **Журнал изменений** (историю)

Что подлежит управлению изменениями:

1. Требования к новой функциональности
2. Нефункциональные требования (производительность, безопасность и т.д.)
3. Исправления дефектов (багов)
4. Артефакты архитектуры, анализа и дизайна

Почему это важно?

Проблема: Когда над проектом работает много людей, изменения накапливаются как "снежный ком". Без контроля возникает хаос.

Решение:

- Changes вносятся **последовательно**, а не одновременно в нескольких местах
- Каждое изменение **фиксируется в журнале**
- Можно отследить **историю правок** и оценить их влияние на систему

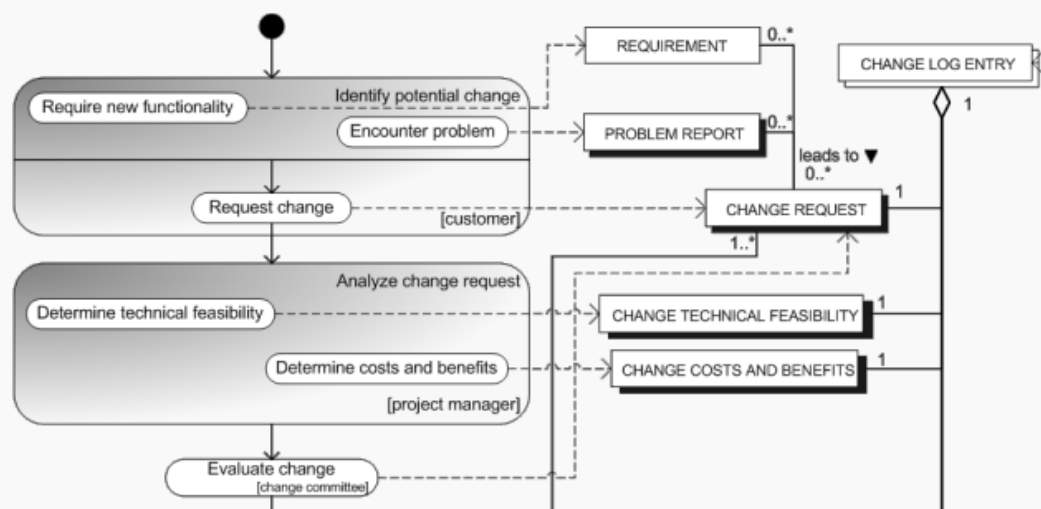
- Контролируемое, журналируемое обновление системы.
- Атрибуты изменения:
 - Идентификатор, дата, ответственный, описание, журнал изменения.
- Управлению изменениями подлежат:
 - Требование новой функциональности.
 - Нефункциональные требование.
 - Исправление дефектов.
 - Другие артефакты архитектуры, анализа, дизайна.

При увеличении количества людей, работающих над проектами, накапливающиеся изменения нарастают как снежный ком, и контроль и управление ими приходится организационно контролировать. Этим занимается дисциплина, называемая управление изменениями (change management). В экономике управление изменениями (например, в организации, ведущей хозяйственную деятельность), является основополагающим инструментом адаптации к меняющимся условиям бизнеса.

У сложных систем (в том числе и программных) невозможно, или очень сложно, менять систему одновременно в нескольких компонентах. Поэтому вносимые изменения обычно выстраиваются в последовательность отдельных действий, которые необходимо контролировать и фиксировать атрибуты каждого такого изменения в журнале.

Системы контроля версий являются примерами систем учета изменений и позволяют вести такие журналы. При помощи журнала можно просмотреть историю правок и оценить их влияние на поведение системы. Каждое изменение имеет атрибуты: идентификатор, дату, ответственного, описание, связанный с ним журнал изменения и т.д.

В разработке ПО не только изменения программного кода могут попадать под контроль — отдельному управлению изменениями обычно подлежат сами требования к ПО, выявленные дефекты, артефакты архитектуры, анализа и дизайна.



[https://en.wikipedia.org/wiki/Change_management_\(engineering\)](https://en.wikipedia.org/wiki/Change_management_(engineering))

На слайде приведена модель процесса управления изменениями, который осуществляется в области разработки систем и, в том числе, в инженерии программного обеспечения. Все современные средства по управлению изменениями реализуют близкий к приведённому процесс.

В левой части указаны активности (деятельности) ролей процесса с целью формирования и изменения набора артефактов для ведения изменения. Справа показаны сами артефакты изменения, участвующие или возникающие в процессе. Следует заметить, что данная модель представляет собой гибрид двух UML-диаграмм — диаграммы деятельности и диаграммы классов верхнего уровня.

Заказчик (Customer) вносит изменения в систему ради появления новой функциональности (Require new functionality) или исправления выявленных им проблем (Encounter problem). Каждая из этих деятельностей приводит к созданию документа или материальных артефактов: Requirements (требования к системе) Problem report (отчёт о проблеме или ошибке). Оба типа запросов формируют т.н. запрос на изменение (Change Request, часто используется акроним CR), и на каждый такой запрос формируются записи в Change Log Entry.

После этого запросы на изменения поступают к менеджеру проекта (Project manager). У менеджера есть две необходимых деятельности, которые он должен осуществить в процессе анализа запроса на изменения: он определяет техническую необходимость и осуществимость изменений (technical feasibility), а также анализирует стоимость и преимущества для пользователя (costs and benefits). При этом первый фактор формирует документ или артефакт Change Technical Feasibility, а второй — Change Costs and Benefits. Всё это отражается в журнале изменений.

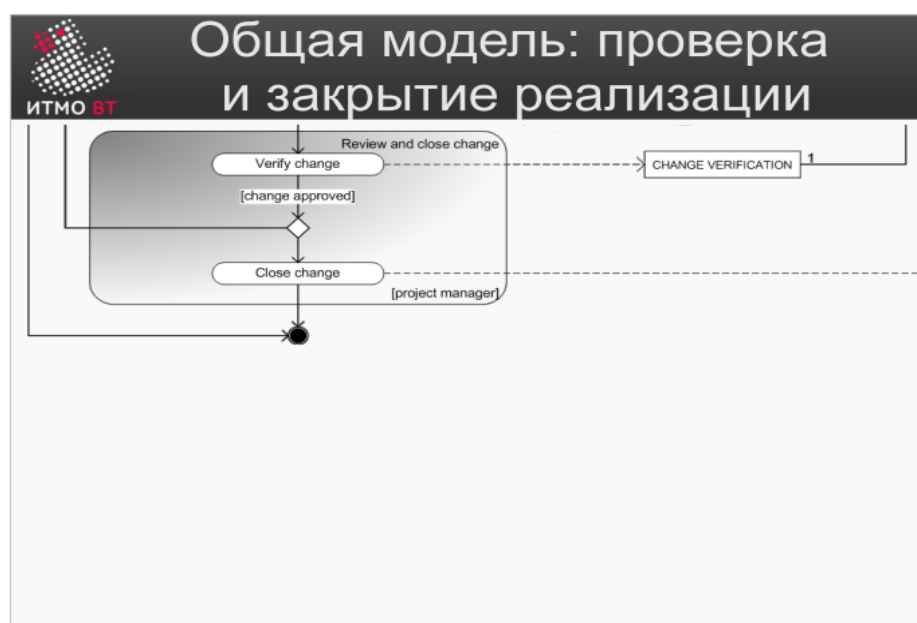
После этого запрос поступает к т.н. комитету по изменениям, который, оценив необходимость изменений и стоимость решения проблемы, меняет статус изменения в Change Request. Если запрос подтверждён, то производится анализ и реализация изменений, в противном случае запрос может быть отменён или отложен.



После принятия решения об изменении анализируется его возможное влияние на другие части системы и возможные последствия для пользователей (Analyze change impact). Необходимо помнить, что в сложных системах внесение изменений в одну часть системы почти всегда приводит к влиянию, и соответственно, изменениям в других частях, поэтому вопрос о влиянии изменения следует тщательно изучить.

После анализа организуется планирование проведения изменений в системе (Create Planning), в ходе которого определяются ресурсы, время и график осуществления действий, необходимых для внесения изменения. В результате этих двух действий появляются артефакты Change Impact Analysis и Change Planning.

Затем изменения передаются на реализацию. Роль, ответственная за неё (Change builder), создает объекты, совокупность которых составляет реализацию изменения (Items), например, исходный код. После этого производится тестирование, вносятся изменения в связанную документацию, и выпускается новый выпуск или версия продукта, «заплатка» и/или конфигурация. При этом создаются артефакты — отчеты о тестировании (Test Report), собственно, документация (Documentation) и выпуск системы (System Release), соответственно.



В конце процесса изменения передаются менеджеру проекта (и иногда — заказчику), проверяются (создаётся артефакт Change Verification), и формально утверждаются.

Эта модель описывает стандартный процесс **управления изменениями (Change Management)** в разработке программного обеспечения и системной инженерии. Она показывает, как правильно обрабатывать запросы на изменение системы, чтобы не сломать работающий продукт и контролировать расходы.

Модель разделена на три логических этапа (соответствующих трем вашим изображениям):

1. Запрос и анализ (Request and Analysis)

Это начальный этап, когда изменение только планируется.

- **Инициатор:** Заказчик (Customer). Он либо хочет новую функцию (*Require new functionality*), либо нашел ошибку (*Encounter problem*).
- **Документирование:** На основе желания заказчика создается формальный **Запрос на изменение (Change Request — CR)**. Также ведется журнал изменений (*Change Log Entry*).
- **Анализ:** Менеджер проекта (Project Manager) изучает запрос по двум направлениям:
 1. **Техническая осуществимость:** Можем ли мы это сделать технически? (*Change Technical Feasibility*).
 2. **Экономика:** Сколько это будет стоить и какую пользу принесет? (*Change Costs and Benefits*).
- **Решение:** Комитет по изменениям (*Change Committee*) рассматривает анализ. Они могут утвердить запрос, отклонить его или отложить.

2. Подтверждение и реализация (Confirmation and Implementation)

Если запрос утвержден, начинается работа над ним.

- **Планирование:** Сначала анализируется влияние изменения на остальную систему (*Analyze change impact* — важно, чтобы новая фича не сломала старую). Затем составляется план работ (*Change Planning*).
- **Разработка:** Роль "Строитель изменений" (*Change Builder* — обычно программист) выполняет работу.
 - Создает объекты кода или конфигурации (*Items*).
 - Выполняет изменение (*Execute change*).
- **Тестирование и выпуск:**
 - Изменение тестируется, создается отчет о тестах (*Test Report*).
 - Обновляется документация (*Documentation*).
 - Выпускается новая версия системы или "заплатка" (*System Release*).

3. Проверка и закрытие реализации (Verification and Closing)

Финальный этап контроля качества.

- **Верификация:** Менеджер проекта (иногда вместе с заказчиком) проверяет, что изменение действительно работает так, как требовалось, и не внесло новых ошибок. Создается артефакт подтверждения (*Change Verification*).
- **Закрытие:** Если все хорошо, статус изменения меняется на "Закрыто" (*Close change*). Процесс завершен.

Главная суть модели: Нельзя просто так взять и поменять код в работающей системе. Нужно сначала оформить запрос, оценить риски и стоимость, получить одобрение, сделать работу, протестировать её и только потом официально закрыть задачу. Это предотвращает хаос в разработке.

1. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ - НАЗНАЧЕНИЕ, ПРИМЕРЫ РЕШЕНИЙ.

<https://gb.ru/blog/sistema-kontrolya-versij/>

Системы контроля версий (СКВ или VCS) разработаны специально для того, чтобы максимально упростить и упорядочить работу над проектом (вне зависимости от того, сколько человек в этом участвуют). СКВ дает возможность видеть, кто, когда и какие изменения вносил; позволяет формировать новые ветви проекта, объединять уже имеющиеся; настраивать [контроль доступа к проекту](#); осуществлять откат до предыдущих версий.

Система контроля версий (VCS — Version Control System) — это инструмент, который сохраняет историю изменений в файлах проекта, позволяя разработчикам отслеживать изменения и управлять версиями кода.

Назначение

VCS помогает:

- **Отслеживать изменения** — фиксировать каждое обновление файлов, указывая, кто и когда внёс правки. Это облегчает поиск ошибок и отладку кода, помогает избежать путаницы и конфликтов.
- **Поддерживать стабильность проекта** — разделять его на версии и облегчать переход от одной версии к другой. Это важно для тестирования и выпуска новых релизов, поскольку позволяет управлять, какая версия проекта используется в производственной среде, а какая — в разработке.
- **Организовывать совместную работу** — несколько разработчиков могут работать над одним проектом одновременно, сливая изменения без конфликтов.
- **Поддерживать параллельную разработку** — система позволяет разработчикам работать над разными функциями и исправлениями одновременно, не мешая друг другу. Это достигается через создание отдельных веток, которые затем сливаются с основной версией проекта.

Виды

Системы контроля версий делятся на типы в зависимости от того, как они организуют хранение и управление версиями:

- **Локальные** — вся история изменений хранится на компьютере разработчика. Все изменения фиксируются в локальном репозитории, и каждый пользователь работает с собственной копией данных. Для обмена файлами с другими разработчиками нужно вручную копировать их.

Локальные СКВ — это простейший тип систем контроля версий, где вся история изменений хранится только на одном компьютере в специальной базе данных. Нет никакого сервера, нет сетевого взаимодействия — всё работает исключительно локально.

- Как это работает?
- Система хранит:
 - Снимки (snapshots) файлов на разные моменты времени
 - Историю изменений каждого файла
 - Метаданные: кто изменил, когда, комментарий к изменению

При изменении файла система сохраняет не всю копию файла, а разницу (diff) между версиями, что экономит место.

- **Централизованные** — хранят всю историю изменений и актуальные версии проекта на одном сервере. Разработчики работают с локальными копиями исходного кода и для получения последних изменений или отправки своих правок подключаются к серверу.

Централизованные СКВ — это системы, где вся история изменений проекта хранится на одном центральном сервере. Все разработчики работают с этим сервером: получают файлы из него, вносят изменения и отправляют их обратно.

- Это как общая библиотека: все книги (файлы проекта) хранятся в одном месте, а читатели (разработчики) приходят туда, чтобы взять книгу, почитать/отредактировать и вернуть обратно.

- **Распределённые** — позволяют каждому разработчику иметь полную локальную копию репозитория с историей изменений. Можно работать без постоянного доступа к серверу и синхронизировать изменения с другими участниками по мере необходимости.

Распределённые СКВ — это системы, где каждый разработчик имеет полную копию репозитория со всей историей изменений. Нет единого центрального сервера, который необходим для работы (хотя серверы часто используются для удобства совместной работы).

Это как если бы каждый член команды имел свою собственную полную библиотеку со всеми книгами и их историей, а не ходил в одну общую.

Ключевое отличие: Каждый разработчик работает с **полноценным репозиторием**, а не просто с рабочей копией файлов.

Репозиторий — это специальное хранилище, где система контроля версий хранит:

- Все файлы вашего проекта
- **Полную историю** всех изменений этих файлов
- Информацию о том, **кто, когда и почему** внёс каждое изменение
- Ветви (branches), теги (tags) и другие метаданные

Примеры

Некоторые примеры систем контроля версий:

- **Git** — распределённая система, используется для множества проектов благодаря своей распределённой природе и гибкости.
- **Subversion (SVN)** — централизованная система, более простая в управлении, но менее гибкая.
- **Mercurial** — похож на Git, но с акцентом на простоту использования и высокую производительность.
- **RCS (Revision Control System)** — позволяет сохранять изменения в локальном репозитории, но не поддерживает работу нескольких разработчиков с одним проектом.

Инструменты

Для работы с системами контроля версий используются, например:

- **Репозиторий** — папка, где лежат файлы проекта и вся история правок. Может быть локальным или удалённым.
- **Коммит (Commit)** — запись об изменении, добавленных в код. Содержит данные о том, кто и когда добавил правки, а также описание изменений.

1. Назначение систем контроля версий (СКВ)

Системы контроля версий (Version Control Systems, VCS) — это программные инструменты, которые позволяют **отслеживать изменения в файлах и проектах с течением времени**. Они особенно полезны при разработке программного обеспечения, работе с документацией или совместной работе над проектами.

Основные цели СКВ:

1. Отслеживание изменений

- Хранение всех версий файлов, чтобы можно было вернуться к любой предыдущей версии.
- Позволяет видеть, кто и когда внес изменения.

2. Совместная работа

- Несколько разработчиков могут работать над одним проектом одновременно.
- Система помогает объединять изменения (мерджить) и разрешать конфликты.

3. История проекта и аудит

- Полная история изменений проекта, включая комментарии к каждому изменению.
- Полезно для анализа ошибок и контроля качества.

4. Ветвление и эксперименты

- Создание «веток» (branches) для новых функций или экспериментов без риска испортить основную версию.
- Позволяет тестировать и внедрять изменения постепенно.

5. Восстановление данных

- В случае ошибок, потери данных или багов можно откатиться к рабочей версии.

2. Примеры систем контроля версий

СКВ делятся на две большие категории:

А. Локальные системы контроля версий

- Хранят версии файлов на одном компьютере.
- Примеры:
 - **RCS (Revision Control System)** — старое решение для отслеживания изменений в отдельных файлах.
 - **SCCS (Source Code Control System)** — более ранняя система, использовавшаяся в UNIX.

В. Централизованные системы контроля версий (CVCS)

- Хранят все изменения на центральном сервере.
- Разработчики получают доступ к последней версии через сервер.
- Примеры:
 - **Subversion (SVN)**
 - **CVS (Concurrent Versions System)**

С. Распределённые системы контроля версий (DVCS)

- Каждый разработчик имеет полную копию репозитория со всей историей.
- Позволяют работать офлайн и потом синхронизировать изменения.
- Примеры:
 - **Git** — самая популярная современная система, используется в GitHub, GitLab, Bitbucket.
 - **Mercurial**



Одновременная модификация файлов

- Lock-modify-unlock:
 - В основном, в СКВ на файловых системах.
 - Замедляет работу команды.
- Copy-modify-merge:
 - Своя рабочая копия у каждого разработчика.
 - Трудности слияния рабочих копий.

Главной проблемой при работе с СКВ является необходимость одновременного редактирования одних и тех же файлов несколькими разработчиками. Одновременное редактирование было технически возможно достаточно давно, но оно приводило к войне правок, когда разработчики в рамках своих заданий вносили взаимно-конфликтующие изменения.

В мире систем контроля версий утвердились два подхода к решению данной проблемы:

1) Подход Lock-modify-unlock, где при работе одного пользователя с файлом, он блокируется для других, и они не могут его модифицировать. Это замедляет работу команды, так как другим пользователям приходится либо ничего не делать, либо переключаться на другие задачи, не связанные с заблокированным файлом. Данный подход был характерен на системах контроля версий, связанных с общей файловой системой.

2) Copy-modify-merge, где каждый пользователь копирует себе весь репозиторий, работает с ним, и изменения всех пользователей сливаются (merge). В данном случае именно слияние рабочих копий разработчиков представляет собой основную проблему. Например, два разработчика могут одновременно изменить один и тот же код двумя различными способами. В этом случае, при попытке сохранить изменения, второй разработчик (тот, который делает коммит по времени вторым, после первого) не сможет этого сделать и будет вынужден разрешать конфликт (во многих случаях — совместно с первым разработчиком).

1. РЕВИЗИИ И ВЕТКИ.

1. Ревизии

Ревизия (commit, версия) — это фиксированное состояние проекта в определённый момент времени.

Основные моменты:

1. Каждая ревизия хранит:
 - **Изменённые файлы**
 - **Комментарий** (commit message), объясняющий, что было сделано
 - **Автора изменений**
 - **Метка времени**
2. Ревизии создаются последовательно и формируют **историю проекта**.
3. Преимущества:
 - Можно **откатиться** к любой прошлой версии.
 - Легко проследить, кто и когда внёс изменения.
 - Позволяет анализировать ошибки и сравнивать версии (diff).

2. Ветки (branches)

Ветка — это отдельная линия разработки внутри проекта.

Основные моменты:

1. Ветка позволяет работать над функцией или исправлением **отдельно от основной версии** (обычно `main` или `master`).
2. Ветки облегчают **параллельную разработку**:
 - Несколько разработчиков могут работать независимо.
 - Основная версия остаётся стабильной.

◆ Взаимосвязь ревизий и веток

- Каждая ветка имеет свою **последовательность ревизий**.
- Слияние веток объединяет **ревизии из разных линий разработки**.

Конфликт — это ситуация, когда Git не может автоматически объединить изменения.

◆ 1. Конфликт слияния (merge conflict)

Возникает, когда:

- два разработчика изменили **один и тот же участок файла**.

📌 Пример:

- в одной ветке изменена строка A
 - в другой ветке — та же строка A
-

◆ 2. Конфликт при rebase

Возникает при использовании rebase, если:

- коммиты не могут быть «перенесены» из-за пересечений изменений.

📌 Особенность:

- конфликты решаются **по одному коммиту**.
-

◆ 3. Конфликт «удаление vs изменение»

Возникает, когда:

- в одной ветке файл удалён,
 - в другой — изменён.
-

◆ 4. Конфликт добавления (add/add)

Возникает, когда:

- в двух ветках добавлен **разный файл с одинаковым именем**.
-

◆ 5. Конфликт переименования

Возникает, когда:

- файл был переименован по-разному в разных ветках.

Способы решения конфликтов

◆ 1. Ручное редактирование

Git помечает конфликт в файле:

```
<<<<<<< HEAD
ваш вариант
=====
чужой вариант
>>>>>>> branch
```

 Нужно:

- выбрать нужный вариант или объединить;
- удалить служебные метки;
- сохранить файл.

◆ 2. Использование инструментов (merge tools)

Специальные программы:

- встроенные или внешние (IDE)

 Помогают:

- визуально сравнить изменения
- выбрать нужную версию

◆ 3. Выбор одной из версий

Можно принять:

- только свою версию
- только чужую версию

 Команды:

- `--ours` (наша версия)
 - `--theirs` (их версия)
-

◆ 4. Отмена операции

Если конфликт слишком сложный:

- отменить merge / rebase:
 - `git merge --abort`
 - `git rebase --abort`

◆ 5. Предотвращение конфликтов (важно для теории)

- чаще синхронизировать ветки
- работать с разными файлами/модулями
- делать небольшие коммиты
- регулярно делать pull/rebase

remote (удалённый) — это **короткое имя (псевдоним)** для ссылки на удалённый репозиторий.

1. Имя по умолчанию — `origin`

Когда вы делаете `git clone`, Git автоматически создаёт remote с именем `origin`, который указывает на тот репозиторий, откуда вы скачали проект.

```
bash
1 # Пример URL, который скрывается за именем "origin"
2 https://github.com/username/project.git
```

2. Зачем это нужно?

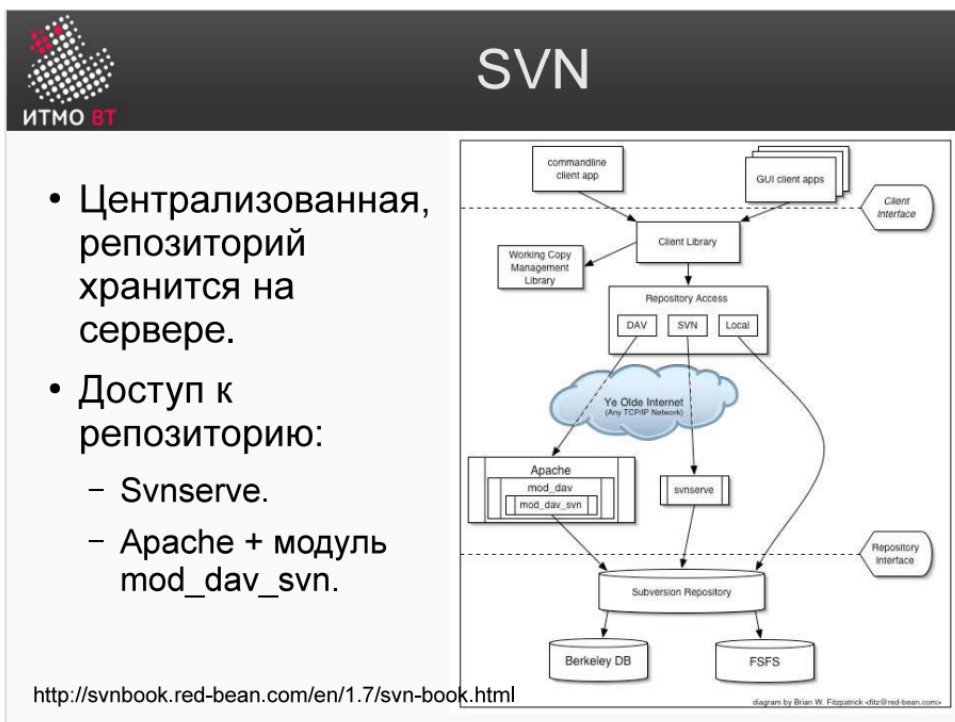
Чтобы не писать длинные URL каждый раз.

Вместо этого:

```
bash
1 git push https://github.com/username/project.git main
```

Вы пишете это:

```
bash
1 git push origin main
```



Одной из популярных систем контроля версий является Subversion (SVN).

В архитектуре, которая приведена на слайде, в существует уровень хранения репозитория (Subversion repository), который может иметь две технические реализации — размещаться в базе данных Berkeley DB, или, в простейшем случае, в файловой системе (FSFS).

Доступом к репозиторию управляет демон svnserve, который получает команды пользователя и выполняет соответствующие изменения в репозитории. Вторым вариантом доступа является использование сервера Apache и его модулей, реализующих ту же логику, что и svnserve.

Удалённый доступ к репозиторию может осуществляться по нескольким протоколам. Обычно используется протокол svn или связки svn+https и ssh+svn. Этими протоколами может пользоваться клиентская библиотека, которую, в свою очередь, используют приложения, интегрирующие в себя поддержку Subversion.

Клиент SVN не только осуществляет передачу данных с репозитория клиенту и обратно. Он ещё и осуществляет управление локальной копией файлов, которую модифицирует разработчик. Это значит, например, что когда вы скачиваете выбранную версию из репозитория, локальный набор файлов должен быть изменён в соответствии с содержимым выбранной версии.

2. Хранение репозитория

Репозиторий SVN может храниться **двумя способами**:

Способ	Описание
Berkeley DB	Специализированная база данных (более сложная, но мощная)
FSFS	Простая файловая система (файлы хранятся как обычные файлы на диске)
FSFS сейчас используется чаще — она проще и надежнее.	

3. Доступ к репозиторию

Есть **два основных способа** управления доступом:

a) svnserve

- Специальный демон (служба/процесс)
- Принимает команды от пользователей
- Выполняет изменения в репозитории
- Работает через протокол [svn://](#)

б) Apache + модули

- Используется веб-сервер Apache
- Специальные модули ([mod_dav_svn](#)) делают то же самое, что `svnserve`
- Работает через HTTP/HTTPS

4. Протоколы доступа

Удаленный доступ может работать по разным протоколам:

- [svn://](#) — собственный протокол SVN (быстрый)
- [svn+https://](#) — через HTTPS (безопасный)
- [ssh+svn://](#) — через SSH (защищенный туннель)

Клиентская библиотека использует эти протоколы для связи с сервером.

5. Клиентская библиотека

Это компонент, который:

- Используют программы с поддержкой SVN
- Например: TortoiseSVN, интеграции в IDE (IntelliJ, Eclipse)
- Скрывает сложность работы с протоколами

6. Что делает SVN клиент

Клиент выполняет **две основные задачи**:

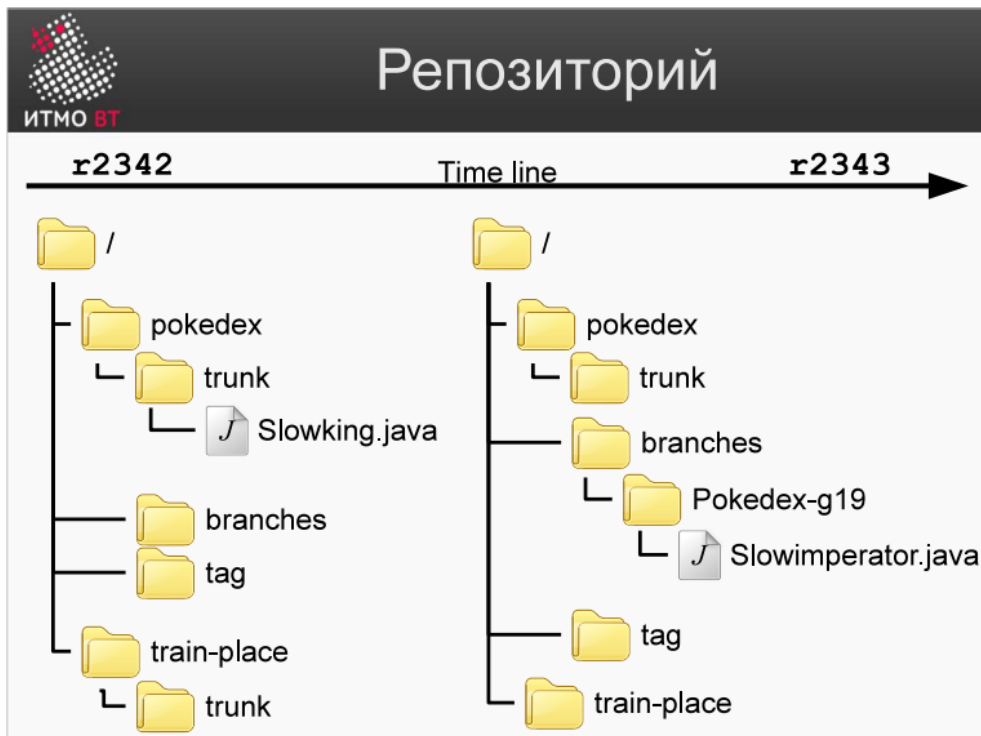
а) Передача данных

- Отправка изменений в репозиторий
- Получение данных из репозитория

б) Управление локальной копией

Когда вы скачиваете определенную версию из репозитория:

- Локальные файлы **изменяются** в соответствии с этой версией
- Клиент сам обновляет, добавляет или удаляет файлы
- Вы видите на своем компьютере именно ту версию, которую выбрали



Репозиторий — это набор файлов проекта(ов), организуемый определённым иерархическим образом для удобной работы с проектом. На слайде показан пример репозитория.

В SVN каждая фиксация изменений (svn commit), пришедшая в репозиторий, повышает версию этого репозитория на 1. Ревизия репозитория — это всегда целое число, уникально характеризующее набор файлов, лежащих в репозитории на какой-то момент времени, задаваемый номером ревизии. В каждый момент времени, при условии успешной фиксации изменений, репозиторий является целостным.

Каждая фиксация изменений svn обязана включать файлы, изменения которых должны быть помещены в репозиторий. Если такое изменение производится из командной строки, то эта строка должна содержать все файлы. В одну фиксацию изменений рекомендуется помещать логически законченное обновление функционала, или, например, исправление одного дефекта. Не рекомендуется объединять разные изменения в один коммит.

Организация файлов обычно такова: существуют каталоги проектов (здесь pokedex и train-place), внутри которых размещены директории, предназначенные для организации работы с разными версиями проекта.

В каталоге trunk происходит основной процесс разработки. В этот каталог разработчики ежедневно вносят результаты своего труда, оформленные в виде целостного набора изменений (с точки зрения и набора файлов, и их логического содержимого). В случае необходимости поставки различных версий разным заказчикам, либо с целью фиксации истории релизов, стабильные на некоторый момент времени копии могут формироваться на основе содержимого каталога trunk и копироваться в дополнительные каталоги branch и tags.

Репозиторий — это организованное хранилище всех файлов проекта(ов) с определенной структурой папок для удобной работы.

12
34

Версионирование в SVN

Ключевые особенности:

- Каждый **svn commit** увеличивает версию репозитория на 1
- Ревизия — это **целое число** (1, 2, 3, ...)

- Каждая ревизия **уникальна** и описывает состояние всех файлов в конкретный момент
- Репозиторий **всегда целостный** (после успешного коммита)



Правила коммитов

Что должно быть в коммите:

✓ Правильно:

- Все измененные файлы указываются в команде commit
- Один коммит = одно логическое изменение
- Примеры:
 - Добавление новой функции
 - Исправление одного бага
 - Рефакторинг одного модуля

✗ Неправильно:

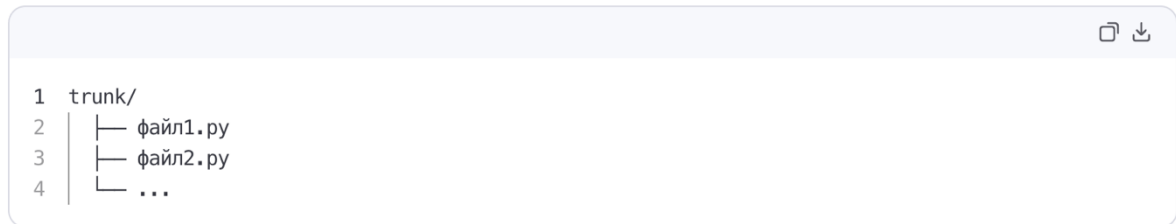
- Объединять разные, не связанные изменения в один коммит
- Пример плохого коммита: "исправил баг + добавил новую функцию + поправил стили"

Рекомендация:

Каждый коммит должен быть **логически законченным** обновлением

1. trunk (ствол)

Основная ветвь разработки — здесь ведется повседневная работа над проектом.



- Все разработчики коммитят изменения сюда
- Здесь самая актуальная (но не всегда стабильная) версия



Шпаргалка

- **trunk** — основная ветвь разработки.
- **branches** — хранение модификаций продукта:
 - Releases — значимые версии продукта (3.0, 3.5).
 - Features — выполнение работ над версиями со существенными изменениями без влияния на trunk.
 - Vendor — версии с модификациями сторонних библиотек. *Vendor drop*.
- **tag** — функционально целостные изменения:
 - Исправления дефектов ПО (3.5.0-B2947219).
 - Минорные версии (с исправлением нескольких дефектов — 3.5.0.1).

trunk — основная ветвь разработки.

В branches хранятся отдельные модификации, в частности, структурно-значимые версии продукта (например, 3.0 или 3.5). Также в branches можно вести разработку отдельных ветвей реализации, в которых внутренний функционал по сравнению с trunk серьёзно изменён, и которые не следует делать в trunk во избежание серьёзных конфликтов. После проведения всех необходимых изменений, данную функциональную реализацию сливают с основной. Существуют также Vendor branches, версии с модификациями сторонних библиотек, используемых в проекте (и, возможно, изменённых), которые необходимо держать вместе с кодом, который их использует.

В каталогах tags хранятся помеченные версии продукта. С точки зрения SVN каталоги tags используются для работы над исправлением дефектов (3.5.0-B2947219), или, например, для минорных версий с исправлением нескольких дефектов (3.5.0.1).

В SVN для создания новой ветки производится полное копирование файлов старой ветки в новый каталог. При этом точно так же всегда увеличивается версия репозитория.

1. trunk (ствол)

Основная ветвь разработки — здесь ведется повседневная работа над проектом.

```
1 trunk/
2 |   ├── файл1.py
3 |   ├── файл2.py
4 |   └── ...
```

- Все разработчики коммитят изменения сюда
- Здесь самая актуальная (но не всегда стабильная) версия

Хранилище модификаций продукта — отдельные направления разработки.

Типы ветвей:

а) Releases (релизы)

- Значимые версии продукта: 3.0, 3.5, 4.0
- Стабильные версии, готовые к использованию

```
1 branches/
2 |   ├── release-3.0/
3 |   ├── release-3.5/
4 |   └── release-4.0/
```

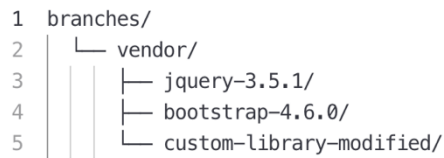
б) Features (функции)

- Разработка **существенных изменений** без влияния на trunk
- Когда нужно долго работать над новой функцией, не ломая основную разработку

```
1 branches/
2 |   ├── feature-new-ui/           ← новая интерфейс
3 |   ├── feature-payment-system/ ← новая система оплаты
4 |   └── feature-analytics/        ← новая аналитика
```

в) Vendor (сторонние библиотеки)

- Версии с модификациями **сторонних библиотек**
- **Vendor drop** — процесс обновления сторонних библиотек



```
1 branches/  
2 |   └─ vendor/  
3 |       ├── jquery-3.5.1/  
4 |       ├── bootstrap-4.6.0/  
5 |       └─ custom-library-modified/
```

3. tags (метки)

Функционально целостные изменения — фиксация конкретных версий.

Для чего используются:

а) Исправления дефектов

- Пример: **3.5.0-B2947219** (версия 3.5.0 с исправлением бага #B2947219)

б) Минорные версии

- Пример: **3.5.0.1** (исправление нескольких дефектов в версии 3.5.0)



Основной цикл разработчика

svn checkout — первоначальное создание рабочей копии.

Ночь! Вечер День Утро

- Обновить рабочую копию:
 - **svn update**
- Изменить необходимые файлы:
 - **svn (add, delete, copy, move, mkdir)**
 - Просмотр изменений: **svn status** и **svn diff**
 - Откат изменений: **svn revert**
- Фиксация изменений на сервере:
 - **svn update** для загрузки изменений других.
 - Разрешить конфликты содержимого и структуры файлов.
 - **svn commit** — собственно фиксация.

Основной цикл разработки использует описанные выше свойства SVN.

В начале работы сервер содержит в рабочей ветке все последние накопленные обновления других разработчиков, и в вашем каталоге существует локальная версия предыдущего дня. Чтобы начать ежедневный цикл работы, необходимо скачать с сервера себе в файловую систему обновление локальной копии. При этом скачиваются не только сами данные, но и метаданные. Управляющая информация нужна для обеспечения целостности и для определения того, куда будет производиться коммит. Таким образом, утром рабочего дня каждый разработчик должен выполнить **svn update** и привести свою рабочую копию до того состояния, которое есть на сервере.

Затем производится "великое творение кода" и меняются необходимые файлы. Используются такие команды, как **svn add** (для создания (добавления) файла), **svn delete** (для удаления файла), **svn copy**, и другие (то же самое касается и директорий, а также перемещения файлов внутри рабочей копии).

Если нужно провести откат изменений, используется команда **svn revert**, а команды **svn status** и **svn diff** помогают ориентироваться в изменениях по сравнению с сервером.

Вечером производится фиксация изменений на сервере. Так как другие разработчики тоже могли внести изменения, с сервера снова скачивается версия, там находящаяся. При этом существует вероятность возникновения конфликтов между изменениями данного разработчика и изменениями, сделанными его коллегами. Данные конфликты необходимо разрешить. После этого производится собственно фиксация с помощью команды **svn commit**.

При этом, тому, кто делает первый коммит, не нужно разрешать каких-либо конфликтов, так как он загружает на сервер первое изменение. Также следует отметить, что конфликты между версиями разных разработчиков могут привести и к конфликтам внутри команды. Существует также понятие ночного билда (Nightly build), который собирается именно на основе вечерних коммитов.



Этапы работы по времени



УТРО — Обновление рабочей копии

Команда: **svn update**

Зачем:

- Скачать все изменения, которые другие разработчики сделали за предыдущий день
- Привести свою локальную копию в актуальное состояние
- Получить не только файлы, но и метаданные SVN

ДЕНЬ — Внесение изменений

1. Изменение файлов

Команды для работы с файлами:

- `svn add file.txt` — добавить новый файл
- `svn delete file.txt` — удалить файл
- `svn copy old.txt new.txt` — копировать файл
- `svn move old.txt new.txt` — переместить/переименовать
- `svn mkdir newdir` — создать директорию

2. Просмотр изменений

Команды для анализа:

- `svn status` — показать какие файлы изменены
- `svn diff` — показать конкретные изменения в коде

3. Откат изменений (если нужно)

Команда: `svn revert file.txt`

Отменяет все локальные изменения в файле, возвращая его к последнему закоммиченному состоянию.

ВЕЧЕР — Фиксация изменений

Шаг 1: Снова обновиться

Команда: `svn update`

Зачем: Перед коммитом нужно скачать изменения, которые другие разработчики могли сделать **сегодня** в течение дня.

Шаг 2: Разрешение конфликтов

Конфликты бывают:

- **Содержимого** — два человека изменили одну и ту же строку кода
- **Структуры** — один изменил файл, другой его удалил

Что делать:

- Открыть конфликтующие файлы
- Вручную выбрать правильную версию
- Отметить конфликт как разрешенный

Шаг 3: Коммит

Команда: `svn commit -m "Описание изменений"`
Финальная отправка всех изменений на сервер.

🌙 НОЧЬ — Nightly Build

Ночная сборка — автоматическая сборка проекта на основе вечерних коммитов.

Зачем:

- Проверить, что все коммиты не сломали сборку
- Найти ошибки интеграции
- Подготовить тестовую версию



Конфликты содержимого файлов

- (e) edit — изменить файл в редакторе.
- (df) diff-full — показать список несовпадений между версиями.
- (r) resolved — подтвердить, что конфликт разрешен в текущей (м.б. отредактированной) версии файла.
- (dc) display-conflict — показать все конфликты.
- (mc) mine-conflict, (tc) theirs-conflict — подтвердить мои (или из репозитория) версии для всех конфликтов.
- (mf) mine-full, (tf) theirs-full — подтвердить версии полностью для файла.
- (p) postpone — отложить «на потом».
- (l) launch — запустить внешнее средство.

```
$ svn update
Updating '.':
U    Pokedex.xml
G    Slowimperator.java
Conflict discovered in 'Slowking.java'
Select:
```

filename.mine — до update
filename.rOLDREV — до правок
filename.rNEWREV
— в репозитории

Основная проблема при фиксации изменений — конфликты содержимого файлов. На слайде приведён пример, в котором при скачивании с сервера возник конфликт между локальной версией разработчика и версией с сервера (например, по-разному поменяли одну и ту же строку).

В случае возникновения конфликта при `svn update` из всех возможных действий чаще всего выбирается (p) postpone — отложить на потом, так как изменения в этот момент редко возможно редактировать самостоятельно. Следует найти разработчика-автора конфликтных изменений, и согласовать с ним исправления.

При выборе postpone также создаются три файла: filename.mine (файл разработчика до update), filename.rOLDREV (файл из репозитория до правок), и filename.rNEWREV (текущий файл в репозитории). Существуют средства (например triple diff), при помощи которых можно удобно сравнить одновременно все три файла.



Конфликты содержимого: Diff

Base (1.8)	4/5	Locally Modified (Based On 1.8)
<pre>} private void loadVersioningSystems(Collection<? extends VersioningSystem> systems) { assert versioningSystems.size() == 0; assert localHistory == null; versioningSystems.addAll(systems); for (VersioningSystem system : versioningSystems) { if (localHistory == null && system instanceof LocalHistory) { localHistory = system; } system.addPropertyChangeListener(this); } } private void unloadVersioningSystems() { for (VersioningSystem system : versioningSystems) { system.removePropertyChangeListener(this); } versioningSystems.clear(); localHistory = null; } InterceptionListener getInterceptionListener() { return filesystemInterceptor; } private synchronized void flushFileOwnerCache() { folderOwners.clear(); }</pre>	<pre>112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139</pre>	<pre>loadVersioningSystems(systems); } private void loadVersioningSystems(Collection<? extends VersioningSystem> systems) { versioningSystems.addAll(systems); for (VersioningSystem system : versioningSystems) { if (localHistory == null && system instanceof LocalHistory) { localHistory = system; } system.removePropertyChangeListener(this); } } private synchronized void unloadVersioningSystems() { for (VersioningSystem system : versioningSystems) { system.removePropertyChangeListener(this); } versioningSystems.clear(); // reset all systems localHistory = null; } InterceptionListener getInterceptionListener() { /** * @return listener for filesystem events */ return filesystemInterceptor; }</pre>

<http://wiki.netbeans.org/NewAndNoteWorthyMilestone8>

Обычно в средствах разработки есть собственные средства Diff для сравнения различных версий. Пример приведён на слайде. В нём различия в коде версий выделены разными цветами. Наиболее глубокого анализа требуют не добавленные или удалённые, а изменённые строки.



Конфликты структуры

Происходят, когда в репозитории файлы перемещаются, а в рабочей копии они же изменяются.

```
$ svn commit -m "Small fixes"
Sending          pokedex/trunk/Slowimperator.java
svn: E155011: Commit failed (details follow):
svn: E155011: File '/pokedex/trunk/Slowimperator.java' is
out of date
svn: E160013: File not found: transaction '14-e', path
'trunk/Slowimperator.java'
$ svn update
Updating '.':
C pokedex/trunk/Slowimperator.java
A pokedex/trunk/SlowImperator.java
Updated to revision 14.
Summary of conflicts:
Tree conflicts: 1
```

Конфликты такого рода происходят тогда, когда, например, в репозитории перемещаются файлы, а в локальной рабочей копии в эти же файлы вносятся изменения. Иными словами, не совпадает структура программ, и файлы, которые были просто отредактированы на локальной копии, в репозитории переименованы, перемещены, или даже удалены.

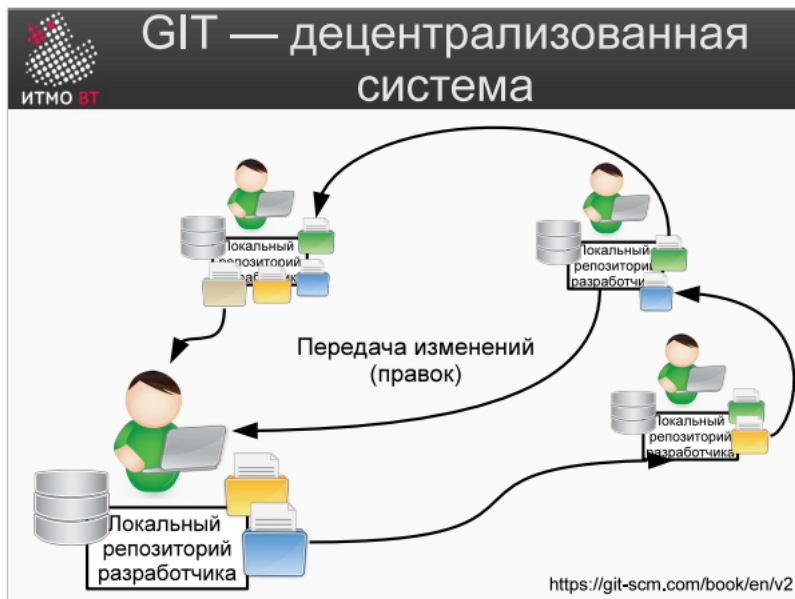
На слайде приведён пример, в котором файл в репозитории был переименован. В локальной копии его имя осталось старым. Конфликт можно разрешить как изменением структуры в локальной копии по образцу версии в репозитории, так и личным общением и согласованием исправлений с тем, кто изменил структуру в репозитории. Не всегда просто определить, в чем заключается конфликт.

📌 Что такое конфликты структуры?

Это более сложный тип конфликтов, который возникает не из-за изменений **внутри** файлов, а из-за изменений **самих** файлов или их **расположения**.

Определение:

Происходят, когда в репозитории файлы **перемещаются/переименовываются/удаляются**, а в рабочей копии эти же файлы **изменяются**.



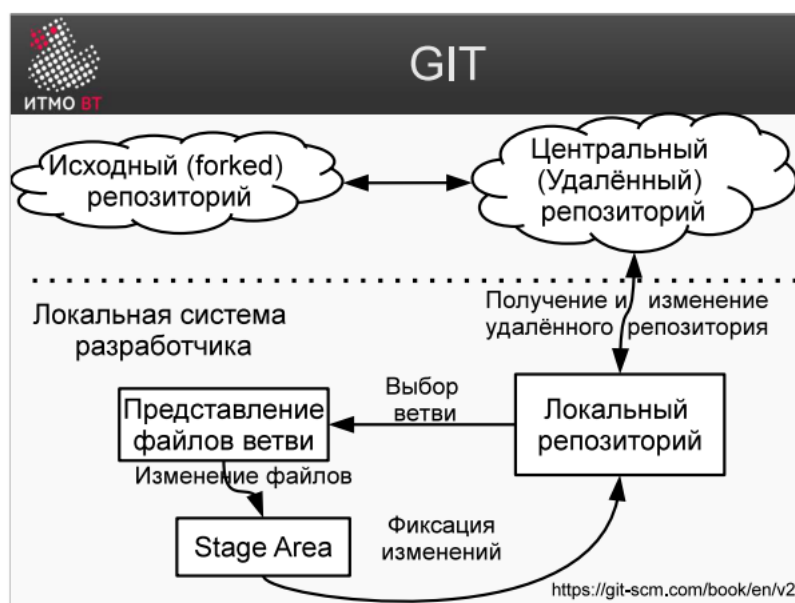
Основной философией одной из самых популярных систем контроля версий — git является ее распределенность. Репозитории существуют локально у каждого разработчика, в них содержится информация о всем проекте целиком. Подчеркнем, что кодовая база, свои изменения и изменения других разработчиков, журналы применения всех доступных изменений находятся у каждого разработчика в локальной копии репозитория проекта.

Когда программист разрабатывает новый функционал (или как говорят на сленге — «фичу»), он может предложить свои правки другим разработчикам, а git предлагает удобный интерфейс по передаче этих правок и внесения их в локальные репозитории коллег-разработчиков. Таким образом может происходить, например, разработка операционных системы — пользователи сами выбирают, какие правки и фичи от других разработчиков им необходимы, чтобы работать над своим функционалом.

Каждый локальный репозиторий имеет свой URL, по которому он может быть доступен, а другие разработчики могут обращаться к нему при помощи алиасов (синонимов), что упрощает команды передачи изменений от разработчика к разработчику и от репозитория к репозиторию.

Благодаря такой архитектуре рабочий процесс совместной работы может быть адаптирован для каждой конкретной команды. В наиболее известном руководстве по Git (ссылка на которое представлена на слайде) определяются три основных рабочих процесса:

- централизованный рабочий процесс — где существует единственный централизованный репозиторий, где все разработчики синхронизируют свою работу с ним;
- рабочий процесс с менеджером по интеграции — где каждый разработчик изменяет канонический репозиторий в своей локальной копии, которую он открывает для других для чтения, а после завершения правок запрашивает интеграционного менеджера загрузить правки из своей копии для чтения в канонический репозиторий;
- рабочий процесс с диктатором и лейтенантами - в основном такой подход используется на огромных проектах, насчитывающих сотни участников; самый известный пример - ядро Linux. Лейтенанты - это интеграционные менеджеры, которые отвечают за отдельные части репозитория. У всех лейтенантов есть свой интеграционный менеджер, который называется доброжелательным диктатором. Репозиторий доброжелательного диктатора выступает как эталонный, откуда все участники процесса должны получать изменения.



Наиболее распространенным в сообществе разработчиков систем с открытым кодом, использующим популярный ресурс GitHub, является рабочий процесс с интеграционным менеджером. В нем существует центральный удалённый репозиторий, в котором сохраняются результаты работы всей команды разработчиков. Центральный репозиторий может быть получен при помощи организации ветви (fork) от любого другого известного проекта.

Такому центральному репозиторию по умолчанию git назначает алиас origin. Центральный репозиторий для каждого из разработчиков команды является удалённым.

У каждого разработчика есть свой локальный репозиторий, который является клоном (копией) удалённого репозитория. Разработчик выбирает необходимую ветвь в локальном репозитории, получая представление на файловой системе файлов с содержимым ветви, которые он может изменять.

Во время работы с рабочей копией разработчик указывает, какие файлы включить в фиксацию изменений. Эти изменения помещаются в т.н. Stage Area — специальную область, из которой файлы будут включены в коммит. В отличие от svn, здесь нет необходимости указывать все файлы, участвующие в фиксации изменения в одной команде — в git их можно добавить за несколько операций. Затем изменения фиксируются в локальном репозитории. Для помещения информации в центральный репозиторий разработчик использует команду push, при которой, естественно, могут возникать конфликты.

Другой способ фиксации изменений в удалённом репозитории — это конфигурация запросов на фиксацию изменений (pull requests). При этом изменения, которые разработчику необходимо записать, сначала должны быть подтверждены владельцем ветви. Таким образом обычно происходит слияние функциональности в открытом программном коде, управляемым сообществом разработчиков. На слайде это показано двусторонней связью "Удалённого репозитория" и "Исходного репозитория".

1. Удалённые репозитории (на сервере)

Центральный репозиторий — официальная версия проекта, куда в итоге попадают все утверждённые изменения. На GitHub это основной репозиторий проекта.

Исходный (forked) репозиторий — ваша личная копия центрального репозитория на сервере. Создаётся через функцию "Fork" на GitHub, когда вы хотите внести вклад в чужой проект.

2. Локальная система разработчика (на вашем компьютере)

Локальный репозиторий — полная копия проекта на вашем компьютере.

Содержит всю историю изменений, все ветки, все метки. Работает без интернета.

Рабочая директория (Working Directory) — файлы проекта, которые вы видите и редактируете в редакторе кода. Это "представление файлов ветви", с которой вы работаете.

Stage Area (Индекс) — промежуточная зона между рабочей директорией и репозиторием. Сюда вы добавляете файлы, которые планируете включить в следующий коммит.

```
git clone https://github.com/user/project.git
```

Создаёт локальную копию удалённого репозитория. Git автоматически назначает алиас **origin** для центрального репозитория.

```
git checkout -b feature-branch
```

Создаёт и переключает на новую ветку для работы.

Редактируете файлы в рабочей директории. Изменения пока не зафиксированы.

```
git add файл.txt
```

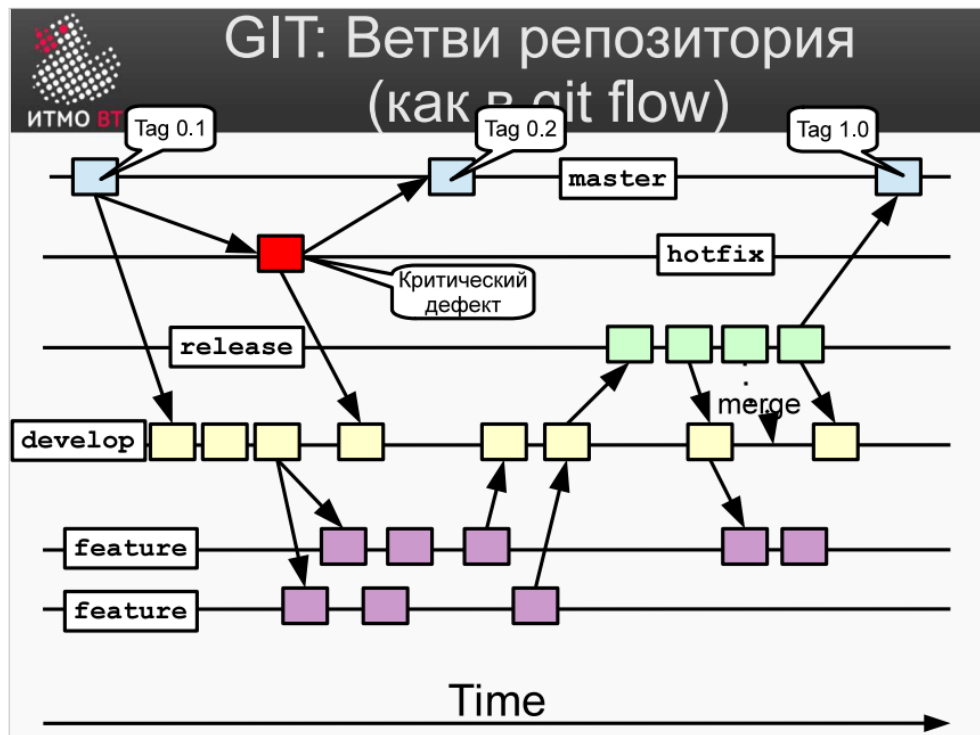
Добавляет изменения в индекс. **Важное отличие от SVN:** в Git можно добавлять файлы частями и формировать коммиты избирательно.

```
git commit -m "Описание изменений"
```

Сохраняет изменения из Stage Area в локальный репозиторий.

```
git push origin feature-branch
```

Отправляет локальные коммиты в удалённый репозиторий.



Модель разработки в git основна на концепции, что каждое различное изменение должно разрабатываться в отдельной ветви. Схематично модель ветвлений представлена на слайде.

В ветви Master находятся версии, готовые к поставке пользователю. Эти версии являются основными версиями продукта. Функционал там чётко определен, и список дефектов известен.

Основная разработка происходит в ветви Develop. Данные ветви используют Master как базовую, и в них происходит разработка новой версии продукта. После кодирования все разработчики сливают свои изменения именно в ветвь Develop. При этом изменения рекомендуется делать по функциональным требованиям продукта. Для каждого из требований создается своя ветвь Feature, и там аккумулируются коммиты для этого требования. По мере завершения разработок функционала коммиты из Feature-ветвей подгружаются в ветвь Develop.

Когда накоплено достаточное количество функционала для выпуска новой, готовой к поставке версии, изменения из Develop формируют ветвь Release, а сама ветвь Develop остаётся для дальнейшей разработки новых функций. Ветвь Release используется для исправления тех дефектов, которые препятствуют выходу новой версии. После исправлений готовая версия продукта копируется в ветвь Master. При этом исправления дефектов сливаются в ветвь Develop, так как они там также будут необходимы в дальнейшем.

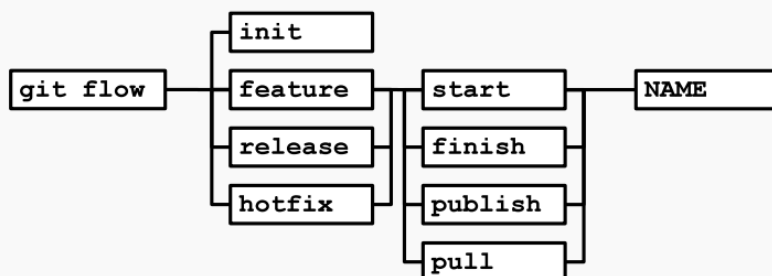
Ветвь Hotfix предназначена для исправления критических ошибок (т.е. тех, которые препятствуют работе) в версии пользователя. После исправления результат передаётся как в ветвь Master, так и в ветвь Develop.

Процесс повторяется для всех новых функциональных требований.



Git flow

- Плагин для GIT — версионирование в терминах в версий, а не операций.



Каждое проделываемое в Git действие над рабочей копией (создание ветвей, их слияние и т.п.) требует достаточно большого количества команд, и может вызвать у разработчиков сложности с синтаксисом команд. Для того, чтобы упростить эту работу, был создан плагин Git flow, позволяющий работать с версиями именно в терминах версий, а не операций, без точного знания команд и последовательностей действий.

Для использования Git flow необходима определённая начальная настройка репозитория. Для этого у Git flow есть функция `init`, которая при старте задаёт пользователю несколько вопросов и производит необходимую настройку.

Для создания новой `feature` в git flow используется команда `git flow feature start FEATURE_NAME`. Когда надо указать на окончание изменений, используется `git flow feature finish FEATURE_NAME`. Остальные команды формируются аналогичным образом. Все это позволяет пользоваться репозиторием на более высоком, независимом от синтаксиса git уровне, при этом реализуется модель, указанная на предыдущих слайдах.

Git Flow — это набор команд-обёрток над стандартным Git.

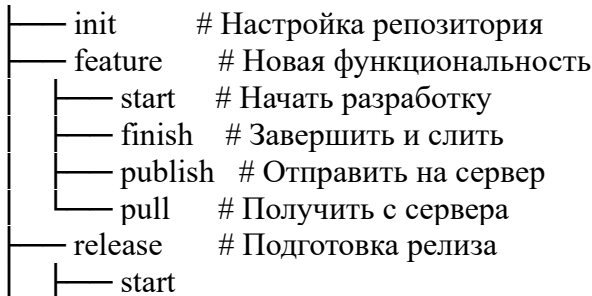
Проблема обычного Git:

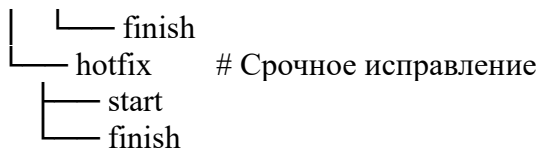
- Нужно помнить много команд (`checkout -b`, `merge`, `branch -d`)
- Легко ошибиться в последовательности действий
- Сложно для новичков

Решение Git Flow:

- Работаете в терминах **задач**, а не команд
- «Начать функцию», «Завершить релиз», «Исправить баг»
- Плагин сам выполняет нужную последовательность команд

git flow





GIT: Шпаргалка по командам

- `git status`
- `git log [--graph]`
- `git diff`
- `git reset --hard HEAD`
- `git branch`
- `git checkout ветвь`
- `git merge ветвь`
- `git commit -m "сообщение"`
- `git add file`

Команды, которые могут понадобиться при выполнении лабораторной работы представлены на слайде. В лабораторной работе функциональность команд сильно урезана, например `git branch` может только показать ветви.

- `git status` — показывает статус текущих состояний файлов в файловой системе и информацию о ветви, в которой производится редактирование.
- `git log` показывает журнал коммитов, а опция `--graph` выводит в графическом виде ветви, в которых производились данные изменения.
- `git diff` — покажет все изменения, которые были сделаны относительно последних зафиксированных изменений.
- `git reset --hard HEAD` сбросит все изменения, которые были сделаны в текущем локальном репозитории.
- `git branch` — показывает ветви.
- `git checkout` — переключает разработчика между ветвями
- `git merge` — объединяет несколько ветвей в текущую ветвь
- `git commit` — фиксирует изменения в текущей ветке. Опция `-m` задает сообщение коммита, которое будет показываться пользователю
- `git add` — добавляет измененные файлы к последующему коммиту, помещая их в Stage Area.

Что такое remote?

`git remote` — это команда для управления ссылками на удалённые репозитории.

Важно понять:

Remote — это не сам репозиторий, а короткое имя-алиас для URL удалённого репозитория.

Аналогия:

Git	Телефонная книга 
Remote (<code>origin</code>)	Контакт "Мама"
URL репозитория	Номер телефона +7-999-...
<code>git push origin</code>	Позвонить "Маме"

Вместо того чтобы каждый раз набирать полный номер (URL), вы используете короткое имя (remote).

Работа с fetch и push

`git fetch` — Получить изменения

```
bash
1 git fetch origin
```



Что делает:

- Скачивает изменения с сервера
- НЕ сливает их с вашей работой
- Обновляет удалённые ветки (`origin/main` , `origin/develop`)

Зачем: Узнать, что изменилось на сервере, прежде чем сливать.

`git pull` — Получить и слить

```
bash
1 git pull origin main
```



Что делает:

- `git fetch` + `git merge`
- Скачивает изменения и сразу сливает с вашей веткой

`git push` — Отправить изменения

```
bash
1 git push origin main
```



Что делает:

- Отправляет локальную ветку `main` в remote `origin`

2. Добавить новый remote

```
bash
```

```
1 git remote add <имя> <URL>
```

Пример:

```
bash
```

```
1 git remote add upstream https://github.com/author/project.git
```

Зачем: Когда работаете с форком:

- `origin` — ваша копия на GitHub
- `upstream` — оригинальный репозиторий автора

🔗 Что такое rebase?

`git rebase` (перебазирование) — это команда для **переноса коммитов** одной ветки на основание другой ветки.

1. Git временно "откладывает" ваши коммиты (A, B, C)
2. Перемещает вашу ветку на новую базу (G)
3. Поочерёдно применяет ваши коммиты поверх новой базы
4. Создаёт **НОВЫЕ** коммиты (A', B', C') с новыми хешами

🔗 Что такое git reset?

`git reset` — это команда для **отмены изменений** и перемещения указателя HEAD на другую позицию в истории коммитов.

Простая аналогия:

Представьте машину времени:

- **Soft** — вернуться назад, но сохранить багаж (изменения в индексе)
- **Mixed** — вернуться назад, взять багаж в руки (изменения в рабочей папке)
- **Hard** — вернуться назад без ничего (все изменения удалены)

Режим	Команда	Индекс	Рабочая папка	Коммиты
Soft	<code>--soft</code>	✅ Сохраняет	✅ Сохраняет	❌ Отменяет
Mixed	<code>--mixed</code> (по умолчанию)	❌ Сбрасывает	✅ Сохраняет	❌ Отменяет
Hard	<code>--hard</code>	❌ Сбрасывает	❌ Удаляет	❌ Отменяет

Команда:

bash



```
1 git reset --soft HEAD~1
```

Что происходит:



1 ДО:

2 [КОММИТ 1] — [КОММИТ 2] — [КОММИТ 3] (HEAD)

[illegible]

5

6 ПОСЛЕ:

7 [КОММИТ 1] – [КОММИТ 2] (HEAD)

```

8 |         ↑
9 |         Изменения в ИНДЕКСЕ (готовы к коммиту)

```


🔑 3. Hard Reset — Жёсткий сброс (ОПАСНО!)

Команда:

```
bash
1 git reset --hard HEAD~1
```

Что происходит:

```
1 ДО:
2 [Коммит 1] — [Коммит 2] — [Коммит 3] (HEAD)
3 |         |         |         |         |
4 |         |         |         |         | ↑
5 |         |         |         |         |   Изменения закоммичены
6
7 ПОСЛЕ:
8 [Коммит 1] — [Коммит 2] (HEAD)
9 |         |         |         |         |
10 |         |         |         |         | ↑
11 |         |         |         |         |   НЕТ НИКАКИХ ИЗМЕНЕНИЙ
12 |         |         |         |         |   (всё удалено безвозвратно!)
```

🔑 2. Mixed Reset — Смешанный сброс (по умолчанию)

Команда:

```
bash
1 git reset HEAD~1
2 # или явно
3 git reset --mixed HEAD~1
```

Что происходит:

```
1 ДО:
2 [Коммит 1] — [Коммит 2] — [Коммит 3] (HEAD)
3 |         |         |         |         |
4 |         |         |         |         | ↑
5 |         |         |         |         |   Изменения закоммичены
6
7 ПОСЛЕ:
8 [Коммит 1] — [Коммит 2] (HEAD)
9 |         |         |         |         |
10 |         |         |         |         | ↑
11 |         |         |         |         |   Изменения в РАБОЧЕЙ ПАПКЕ (нужен git add)
```



Что такое remote

👉 **Remote (удалённый репозиторий)** — это другой репозиторий, который находится:

- на сервере (например GitHub)
- или у другого разработчика

📌 У него есть URL:

`https://github.com/user/project.git`



Зачем нужен `git remote`

Чтобы:

- отправлять изменения (`git push`)
- получать изменения (`git pull`)
- работать в команде



Основные команды



Посмотреть список `remote`

```
git remote -v
```

👉 Покажет:

- имена (например `origin`)
- ссылки (URL)



Добавить `remote`

```
git remote add origin URL
```

👉 Пример:

```
git remote add origin https://github.com/user/project.git
```

📌 `origin` — это просто имя (можно любое, но обычно используют это)



Удалить `remote`

```
git remote remove origin
```



Изменить URL

```
git remote set-url origin NEW_URL
```



Как это выглядит

Твой компьютер ↔ origin (GitHub)

👉 origin = “адрес, куда ты пушишь”



Важный момент

Когда ты пишешь:

```
git push origin main
```

это значит:

- origin → куда отправлять
- main → какую ветку



Аналогия

git remote — это как **контакты в телефоне** 📱

- имя: origin
- номер: URL репозитория

Ты не запоминаешь URL — используешь имя



Коротко

- git remote → список подключённых репозиториев
- origin → стандартное имя главного репозитория
- нужен для push / pull

Если хочешь, могу:

- объяснить разницу **origin vs upstream (очень важно)**
- или разобрать реальный пример с GitHub шаг за шагом

не понимаю

Окей, давай вообще с нуля и максимально просто, без терминов 📌



Представь ситуацию

У тебя есть проект на компьютере:

`my-project`

И есть сайт (например GitHub), где тоже лежит этот проект.

👉 Нужно как-то связать твой проект с сайтом



Вот это и делает `git remote`

👉 `git remote` = “связь с другим репозиторием”



Очень простая аналогия

Представь:

- у тебя есть телефон 📱
- ты сохраняешь контакт:

Имя: `origin`

Телефон: `https://github.com/user/project.git`

👉 Теперь вместо длинного URL ты просто говоришь: **origin**



Что происходит в реальности

Ты один раз делаешь:

`git remote add origin https://github.com/user/project.git`

👉 Это значит:

“Мой проект теперь связан с этим репозиторием”



И потом используешь

Отправить изменения:

```
git push origin main
```

👉 перевод:

“Отправь мою ветку `main` в тот репозиторий, который называется `origin`”

Получить изменения:

```
git pull origin main
```

👉 перевод:

“Забери изменения из `origin`”



Самое главное

👉 `origin` — это НЕ команда

👉 это просто **имя удалённого репозитория**



Коротко

- `git remote` → список “связей”
- `origin` → имя этой связи
- нужен, чтобы:
 - отправлять (`push`)
 - получать (`pull`)